

Backend Developer Assignment

Three-Way Match Engine for PO, GRN, and Invoice

Build a backend service that allows users to upload **Purchase Order (PO)**, **Goods Receipt Note (GRN)**, and **Invoice** documents, extract structured data using **Gemini API**, store the extracted data in **MongoDB**, and perform a **three-way match**.

This assignment is intentionally scoped to be completed in **2-3 days**.

Required Stack

Use the following:

- **Node.js**
- **Express**
- **MongoDB**
- **Gemini API**

You may use any supporting libraries that you think are appropriate.

Objective

Design and implement a backend where:

- the user uploads PO, GRN, and Invoice files
- each document is parsed using Gemini into structured JSON
- the parsed JSON is stored in MongoDB
- documents are linked using `poNumber`
- matching should work even if documents arrive in **any order**
- for every `poNumber` :
 - there is **only 1 PO**
 - there can be **multiple GRNs**

- there can be **multiple Invoices**
-

Matching Rules

Implement the following validations:

- **GRN quantity must not be greater than PO quantity**
- **Invoice quantity must not be greater than total GRN quantity**
- **Invoice quantity must not be greater than PO quantity**
- **Invoice date must not be after PO date**

You should perform matching at the **item level**.

You may choose your own item matching key, but you must explain your choice briefly in the README.

Out-of-Order Upload Requirement

The order of document arrival must **not matter**.

Examples:

- Invoice may arrive before PO
- GRN may arrive before Invoice
- PO may arrive last

Your system should:

- store parsed documents independently
 - attempt matching whenever related documents become available
 - return the latest match state for a given `poNumber`
-

Minimum APIs to Build

Implement at least these APIs:

1. Upload document

`POST /documents/upload`

Input:

- file
- `documentType( po , grn , invoice )`

Behavior:

- upload the file
- parse it using Gemini
- store parsed output
- trigger or update matching logic

2. Get parsed document

`GET /documents/:id`

Returns stored parsed document.

3. Get three-way match result by PO number

`GET /match/:poNumber`

Returns:

- current documents linked to that PO number
 - match status
 - mismatch reasons, if any
-

Minimum Data to Extract

Your structured JSON should contain at least the following:

PO

- poNumber
- poDate
- vendorName
- items[]
 - itemCode or sku
 - description
 - quantity

GRN

- grnNumber
- poNumber
- grnDate
- items[]
 - itemCode or sku
 - description
 - receivedQuantity

Invoice

- invoiceNumber
- poNumber
- invoiceDate
- items[]
 - itemCode or sku
 - description
 - quantity

You may extract more fields if you want, but this is enough for the assignment.

Expected Match Output

For a given `poNumber` , return a result such as:

- `matched`
- `partially_matched`
- `mismatch`
- `insufficient_documents`

Also include reasons where applicable, such as:

- `grn_qty_exceeds_po_qty`
- `invoice_qty_exceeds_po_qty`

- `invoice_qty_exceeds_grn_qty`
 - `invoice_date_after_po_date`
 - `duplicate_po`
 - `item_missing_in_po`
-

Scope Guidance

This is a **backend design and problem-solving task**, not a polished production build.

Focus on a clean, sensible implementation.

Deliverables

Please create a **public GitHub repository** and share the repo link.

Your repository must include:

1. Working code

A backend project that can be run locally.

2. README.md

A concise README explaining:

- your approach
- data model
- parsing flow
- matching logic
- how you handled out-of-order uploads
- assumptions
- tradeoffs
- what you would improve with more time

3. API usage examples

Provide one of:

- Postman collection

- Swagger/OpenAPI

5. Example outputs

Include:

- sample parsed JSON
- sample match result